

$Z(m)$ – это результат свертки или корреляции.

$X(h)$ это первый сигнал, с которым мы работаем.

m – текущий индекс в результирующем массиве $Z(m)$.

h – индекс текущего элемента первого сигнала.

$Y(m-h)$ – это элемент второго сигнала, сдвинутый назад на h .

Дискретное преобразование Фурье (DFT) - преобразует сигнал из временной области в частотную, показывая, какие частоты присутствуют в сигнале. В коде реализована функция `dft()` для прямого преобразования сигнала в его частотное представление.

Обратное преобразование Фурье (IDFT) - восстанавливает сигнал из его частотного представления обратно во временную область. В программе используется функция `idft()` для обратного преобразования после операций в частотной области.

Свертка – это математический способ комбинирования двух сигналов для формирования третьего сигнала. Это один из самых важных методов ЦОС. Пользуясь стратегией импульсного разложения, системы описываются сигналом, называемым импульсной характеристикой. Свертка важна, так как она связывает три сигнала: входной сигнал, выходной сигнал и импульсную характеристику. – по методу.

Почему переворачиваем фильтр? Потому что в математике свертка означает "реакцию системы" на входной сигнал, а реакция получается от перевернутого шаблона. При применении свертки - получаем сглаженный сигнал, в котором уменьшились резкие перепады. Сглаживаем сигнал X по фильтру Y . Применение: фильтрация сигналов, обработка изображений (размытие, выделение краев).

$$Z(m) = \frac{1}{N} \sum_{h=0}^{N-1} X(h)Y(m-h)$$

Свертка - операция, показывающая как одна функция "накладывается" на другую при различных сдвигах. В коде реализована в функции `convolution()` для демонстрации работы линейных систем и фильтрации.

Корреляция - мера сходства между двумя сигналами при разных временных сдвигах. Функция `correlation()` используется для сравнения сигналов и поиска временных задержек между ними. Корреляция дает максимум в момент, когда сигналы совпадают. Почему не переворачиваем? Потому что нам важно найти похожий участок, а не применять фильтр. Применение: Поиск шаблонов в сигнале, распознавание речи, обработка изображений.

$$\hat{Z}(m) = \frac{1}{N} \sum_{h=0}^{N-1} X(h)Y(m+h)$$

Автокорреляция - частный случай корреляции, когда сигнал сравнивается сам с собой. В коде применяется для проверки периодичности сигнала и анализа его спектральных свойств.

Частота дискретизации (F_s) - количество измерений сигнала в секунду. В программе установлена 100 Гц и определяет шаг дискретизации при генерации тестовых сигналов.

Теорема о свертке - утверждает, что свертка во временной области соответствует умножению в частотной. Используется в функциях `fft_convolution()` и `fft_correlation()` для ускорения вычислений.

Быстрое преобразование Фурье (FFT) - алгоритм для быстрого вычисления ДПФ. Хотя в коде используется DFT для наглядности, на практике следует применять FFT для производительности.

Нормализация сигнала - приведение сигнала к нужному диапазону значений. Реализована в функции `save_to_wav()` перед записью в аудиофайл.

WAV-файл - цифровой аудиоформат для хранения звуковых данных. В коде используется для сохранения сгенерированных сигналов и результатов обработки.

Комплексная экспонента - математическая функция, используемая в ДПФ. Применяется в функциях `dft()` и `idft()` для представления гармонических составляющих.

Вещественная и мнимая части - компоненты комплексного числа. В коде используются для работы с частотным представлением сигналов и их визуализации.

Линейная свертка - тип свертки, где длина результата равна сумме длин входных сигналов минус один. Реализована в функции `convolution()` как базовый алгоритм обработки.

Круговая свертка - свертка, при которой сигналы считаются периодическими. Возникает при умножении ДПФ сигналов и учитывается в функциях частотной свертки.

Сдвиг в свертке нужен, потому что мы анализируем, как один сигнал взаимодействует с перевернутой копией другого сигнала при разных положениях. Почему сдвиг от 0 до $N+M-1$? - Начинаем с минимального перекрытия (левый край одного сигнала касается правого края другого). Затем Y двигается вправо, пока полностью не пройдет по X . Так как свертка должна учитывать каждый возможный сдвиг, результат получается длины $N+M-1$.

В `convolution` и `correlation` нет $1/N$? - т.к. нормализуем в `dft()`.

Разница между сверткой во временной области и сверткой через Фурье

Во временной области:

$$Z(m) = \frac{1}{N} \sum_{h=0}^{N-1} X(h)Y(m-h)$$

Здесь свертка выполняется непосредственно через сумму произведений элементов массива. Время выполнения $O(N^2)$, потому что приходится перебирать все элементы в двойном цикле.

Если через Фурье: Берем DFT от обоих сигналов. Умножаем их в частотной области. Делаем обратное DFT для получения результата во временной области.

Что было бы без `fftshift(z)`?

Без этого нулевая частота находилась бы не в центре спектра, а на краю, что делает интерпретацию сложнее.

$L=N+M-1$, где L – это длина результирующего сигнала после свертки. Без умножения на L ($z = \text{idft}(Z) * L$) результат был бы ослабленным по амплитуде. Почему -1 ? – Каждое новое слагаемое добавляется к уже имеющимся, но начальный элемент двух массивов, при наложении Y на X уже присутствует. Поэтому итоговая длина меньше на 1.

`np.fft.fftshift(z)`

FFT представляет частоты в следующем порядке:

Левая часть массива содержит низкие частоты.

Правая часть массива содержит высокие частоты.

Но при этом центральная частота (нулевая) оказывается в начале массива. `fftshift` делает так, чтобы нулевая частота оказалась в центре.

```
# Импорт необходимых библиотек

import numpy as np # Библиотека для численных вычислений и работы с массивами

import matplotlib.pyplot as plt # Библиотека для визуализации данных

from scipy.io import wavfile # Модуль для работы с WAV-файлами

from scipy.signal import convolve # Функция свертки из SciPy
(импортирована, но не используется)
```

```
"""
```

Функция прямого дискретного преобразования Фурье (DFT)

Реализует формулу: $X[k] = (1/N) * \sum_{n=0}^{N-1} x[n] * \exp(-2\pi j * k * n / N)$

где:

- $X[k]$ - комплексный спектр на частоте k
- $x[n]$ - входной сигнал
- N - длина сигнала
- j - мнимая единица

Принцип работы: преобразует сигнал из временной области в частотную

Сложность: $O(N^2)$ - квадратичная, из-за вложенных циклов

```
"""
```

```
def dft(x):
```

```
    N = len(x) # Определяем длину входного сигнала
```

```
    X = np.zeros(N, dtype=complex) # Создаем массив для хранения
    комплексного спектра
```

```
    for k in range(N): # Цикл по всем частотам
```

```
        for n in range(N): # Цикл по всем временным отсчетам
```

```
            # Вычисляем вклад каждого отсчета в каждую частотную
            компоненту
```

```
            X[k] += x[n] * np.exp(-2j * np.pi * k * n / N)
```

```
    return X/N # Возвращаем нормированный спектр
```

```
"""
```

Функция обратного дискретного преобразования Фурье (IDFT)

Реализует формулу: $x[n] = \sum_{k=0}^{N-1} X[k] * \exp(2\pi j * k * n / N)$

Принцип работы: преобразует спектр обратно во временную область

Сложность: $O(N^2)$ - аналогично прямому преобразованию

"""

```
def idft(X):
```

```
    N = len(X) # Определяем длину спектра
```

```
    x = np.zeros(N, dtype=complex) # Создаем массив для
восстановленного сигнала
```

```
    for n in range(N): # Цикл по всем временным отсчетам
```

```
        for k in range(N): # Цикл по всем частотам
```

```
            # Восстанавливаем сигнал как сумму вкладов всех частотных
компонент
```

```
            x[n] += X[k] * np.exp(2j * np.pi * k * n / N)
```

```
    return x # Возвращаем восстановленный сигнал
```

"""

Функция линейной свертки двух сигналов

Реализует формулу: $(x * y)[n] = \sum_{m} x[m] * y[n-m]$

где:

- x, y - входные сигналы

- n - индекс в результирующем сигнале

Принцип работы: вычисляет выходной сигнал как сумму произведений
одного сигнала

на перевернутый и сдвинутый другой сигнал

Сложность: $O(N*M)$, где N и M - длины входных сигналов

"""

```
def convolution(x, y):
```

```
    n = len(x) # Длина первого сигнала
```

```
    m = len(y) # Длина второго сигнала
```

```
    # Создаем массив для результата (длина N+M-1)
```

```
    result = np.zeros(n + m - 1, dtype=np.float64)
```

```
    for i in range(n+m-1): # Для каждого элемента результата
```

```

    for j in range(m): # Для каждого элемента второго сигнала
        if 0 <= i - j < n: # Проверка границ массива
            # Накопление суммы произведений
            result[i] += x[i-j] * y[j]
    return result

```

$$Z(m) = \frac{1}{N} \sum_{h=0}^{N-1} X(h)Y(m-h)$$

"""

Функция кросс-корреляции двух сигналов

Реализует формулу: $(x * y)[n] = \sum (\text{по } m) x[m] * y[m+n]$

Принцип работы: измеряет степень сходства между сигналами при различных временных сдвигах

Сложность: $O(N*M)$, где N и M - длины входных сигналов

"""

```

def correlation(x, y):
    n = len(x) # Длина первого сигнала
    m = len(y) # Длина второго сигнала
    # Создаем массив для результата (длина N+M-1)
    result = np.zeros(n + m - 1, dtype=np.float64)
    for i in range(n): # Для каждого элемента первого сигнала
        for j in range(m): # Для каждого элемента второго сигнала
            # Вычисляем корреляцию с учетом сдвига
            result[i - j + (m - 1)] += x[i] * y[j]
    return result

```

$$\hat{Z}(m) = \frac{1}{N} \sum_{h=0}^{N-1} X(h)Y(m+h)$$

Параметры сигналов для генерации

Fs = 100 # Частота дискретизации в Герцах (количество отсчетов в секунду)

duration = 1 # Длительность сигнала в секундах

```
"""
```

Создание временной оси:

- от 0 до $2\pi \cdot \text{duration}$ (один полный период для тригонометрических функций)
- количество точек: $F_s \cdot \text{duration}$ (частота дискретизации * длительность)
- `endpoint=False` исключает конечную точку из интервала

```
"""
```

```
t = np.linspace(0, 2 * np.pi * duration, int(Fs * duration),  
endpoint=False)
```

```
# Параметры генерируемых сигналов
```

```
f1 = 1 # Частота синусоидального сигнала (1 Гц)
```

```
f2 = 4 # Частота косинусоидального сигнала (4 Гц)
```

```
# Генерация сигналов
```

```
signal_sin = np.sin(f1 * t) # Синусоида с частотой f1
```

```
signal_cos = np.cos(f2 * t) # Косинусоида с частотой f2
```

```
"""
```

Визуализация сгенерированных сигналов:

Создаем график с двумя подграфиками (2 строки, 1 столбец)

```
"""
```

```
plt.figure(figsize=(12, 6)) # Создаем фигуру размером 12х6 дюймов
```

```
# Первый подграфик - синусоида
```

```
plt.subplot(2, 1, 1) # 2 строки, 1 столбец, первый график
```

```
plt.plot(t, signal_sin, label=f'sin(2π*{f1}t)') # Рисуем график  
синусоиды
```

```
plt.title('Сигнал sin') # Заголовок
```

```
plt.xlabel('Время (сек)') # Подпись оси X
```

```
plt.xlim(0, 2*np.pi) # Устанавливаем границы оси X (один полный  
период)
```

```
plt.grid() # Включаем сетку
```

```

# Второй подграфик - косинусоида
plt.subplot(2, 1, 2) # 2 строки, 1 столбец, второй график
plt.plot(t, signal_cos, label=f'cos(2π*{f2}t)') # Рисуем график
косинусоиды
plt.title('Сигнал cos') # Заголовок
plt.xlabel('Время (сек)') # Подпись оси X
plt.xlim(0, 2*np.pi) # Устанавливаем границы оси X (один полный
период)
plt.grid() # Включаем сетку

plt.tight_layout() # Автоматическая подгонка расположения графиков
plt.show() # Отображаем графики

"""
Функция сохранения сигнала в WAV-файл
Аргументы:
- signal: массив отсчетов сигнала
- filename: имя файла для сохранения
- Fs: частота дискретизации
"""

def save_to_wav(signal, filename, Fs):
    # Нормализация сигнала к 16-битному целочисленному формату (-32768
до 32767)
    signal_normalized = np.int16((signal / np.max(np.abs(signal))) *
32767)
    # Запись в WAV-файл
    wavfile.write(filename, Fs, signal_normalized)

# Сохранение сгенерированных сигналов в WAV-файлы
save_to_wav(signal_sin, 'signal_sin.wav', Fs) # Сохраняем синусоиду
save_to_wav(signal_cos, 'signal_cos.wav', Fs) # Сохраняем косинусоиду

```



```
# Вычисление свертки и корреляции наших сигналов
conv_result = convolution(signal_sin, signal_cos) # Свертка sin и cos
corr_result = correlation(signal_sin, signal_cos) # Корреляция sin и cos

"""
Визуализация результатов свертки и корреляции:
Создаем график с двумя подграфиками (2 строки, 1 столбец)
"""

plt.figure(figsize=(12, 6)) # Создаем новую фигуру

# Первый подграфик - результат свертки
plt.subplot(2, 1, 1)
plt.plot(conv_result, label='Свертка sin и cos')
plt.title('Результат свертки')
plt.grid()

# Второй подграфик - результат корреляции
plt.subplot(2, 1, 2)
plt.plot(corr_result, label='Корреляция sin и cos')
plt.title('Результат корреляции')
plt.grid()

plt.tight_layout() # Автоматическая подгонка
plt.show() # Отображаем графики

# Сохранение результата свертки в WAV-файл
save_to_wav(conv_result, 'conv_result.wav', Fs)
```

"""

Функция свертки через преобразование Фурье - свертка во временной области = умножение в частотной области Алгоритм:

1. Дополняем сигналы нулями до длины $N+M-1$
2. Вычисляем DFT обоих сигналов
3. Перемножаем спектры
4. Вычисляем обратное DFT
5. Масштабируем результат

Сложность: $O((N+M) \log(N+M))$ при использовании FFT

"""

```
def fft_convolution(x, y):
    n = len(x) # Длина первого сигнала
    m = len(y) # Длина второго сигнала
    L = n + m - 1 # Минимальная длина для линейной свертки
    # Дополняем сигналы нулями до длины L
    x_pad = np.pad(x, (0, L - n))
    y_pad = np.pad(y, (0, L - m))
    # Вычисляем DFT обоих сигналов
    X = dft(x_pad)
    Y = dft(y_pad)
    # Поэлементное умножение спектров
    Z = X * Y #  $C_z(k) = C_x(k) * C_y(k)$ 
    # Обратное преобразование Фурье
    z = idft(Z)
    # Масштабирование результата
    z = z * L
    # Возвращаем вещественную часть (мнимая должна быть близка к нулю)
    return np.real(z)
```

$$C_z(k) = C_x(k)C_y(k)$$

"""

Функция корреляции через преобразование Фурье

Использует свойство: корреляция = свертка с сопряженным и отраженным сигналом. Алгоритм аналогичен свертке, но использует комплексное сопряжение

"""

```
def fft_correlation(x, y):  
    n = len(x) # Длина первого сигнала  
    m = len(y) # Длина второго сигнала  
    L = n + m - 1 # Минимальная длина  
    # Дополняем сигналы нулями  
    x_pad = np.pad(x, (0, L - n))  
    y_pad = np.pad(y, (0, L - m))  
    # Вычисляем DFT  
    X = dft(x_pad)  
    Y = dft(y_pad)  
    # Комплексное сопряжение второго спектра  
    Y_conj = np.conj(Y)  
    # Поэлементное умножение  
    Z = X * Y_conj  
    # Обратное преобразование  
    z = idft(Z)  
    # Сдвиг результата для центрирования  
    z = np.fft.fftshift(z)  
    # Масштабирование  
    z = z * L  
    # Возвращаем вещественную часть  
    return np.real(z)
```

$$C_z(k) = C_x(k)C_y^*(k)$$

```

# Вычисление свертки и корреляции через Фурье
fft_conv_result = fft_convolution(signal_sin, signal_cos)
fft_corr_result = fft_correlation(signal_sin, signal_cos)

"""
Визуализация результатов свертки и корреляции через Фурье:
Создаем график с двумя подграфиками (2 строки, 1 столбец)
"""

plt.figure(figsize=(12, 6))

# Первый подграфик - свертка через Фурье
plt.subplot(2, 1, 1)
plt.plot(fft_conv_result, label='Свертка через Фурье')
plt.title('Свертка через Фурье')
plt.grid()

# Второй подграфик - корреляция через Фурье
plt.subplot(2, 1, 2)
plt.plot(fft_corr_result, label='Корреляция через Фурье')
plt.title('Корреляция через Фурье')
plt.grid()
plt.tight_layout()
plt.show()

# Сохранение результата свертки через Фурье в WAV-файл
save_to_wav(fft_conv_result, 'fft_conv_result.wav', Fs)

# Вывод информации о вычислительной сложности алгоритмов
print('Оценка сложности:')
print("Сложность свертки:  $O(N*M)$ ") # Для прямой реализации

```

```
print("Сложность корреляции:  $O(N*M)$ ") # Для прямой реализации

# Демонстрация автокорреляции и автопроверки
autoconv = convolution(signal_sin, signal_sin) # Свертка sin с самим собой

# Визуализация автокорреляции
plt.figure()
plt.plot(autoconv, label='Свертка sin с собой')
plt.title('Свертка сигнала sin с собой')
plt.grid()
plt.show()

# Вычисление автокорреляции
autocorr = correlation(signal_sin, signal_sin)

# Визуализация автокорреляции
plt.figure()
plt.plot(autocorr, label='Автокорреляция sin')
plt.title('Автокорреляция сигнала sin')
plt.grid()
plt.show()

# Дополнительная проверка - автокорреляция через свертку
autocorr = convolution(signal_sin, signal_sin)

# Визуализация результата
plt.figure()
plt.plot(autocorr, label='Автокорреляция sin')
plt.title('Автокорреляция сигнала sin')
plt.grid()
plt.show()
```